

Factory Manager — Desktop Application

PROJECT REPORT

1. Introduction

Factory Manager is a desktop application built entirely in Python, designed to help small manufacturing businesses manage their operations digitally. The application was developed for a wooden box manufacturing business that produces packaging for various industries including electronics and marine equipment.

The application provides modules for inventory management, sales tracking, invoice generation, delivery monitoring, and report generation. It runs as a native desktop window on Windows using the CustomTkinter GUI library, and stores all data in a local SQLite database ensuring data is always available without any internet connection.

2. Problem Statement

Small manufacturing businesses often rely on manual methods such as handwritten registers or spreadsheets to track inventory, sales, and deliveries. These methods are error-prone, time-consuming, and difficult to manage as the business grows.

The specific problems identified in the target business were:

- No system to track raw material stock levels or receive low-stock warnings.
- Sales records maintained manually leading to errors and loss of data.
- No system for generating professional invoices for customers.
- Delivery status tracked informally with no organized record.
- No way to generate quick reports for daily or monthly review.

3. Objectives

- To develop a desktop application for factory management using Python.
- To eliminate manual record-keeping by providing a digital inventory and sales system.

- To generate professional PDF invoices automatically.
- To track delivery status of customer orders in real time.
- To produce daily and monthly reports exportable to Excel.
- To package the application as a standalone .exe file for easy distribution.
- To provide both Dark and Light mode for better user experience.

4. Technology Used

Technology	Details
Language	Python 3.x
GUI Framework	CustomTkinter
Database	SQLite3 (built-in)
PDF Generation	fpdf2
Excel Export	pandas + openpyxl
Packaging	PyInstaller (.exe)
IDE	Visual Studio Code
Platform	Windows

5. System Architecture

The application follows a modular architecture with clear separation of concerns:

app.py — Main entry point. Initializes the database, builds the window, sidebar navigation, and loads all pages.

config.py — Centralized configuration file containing database path, app name, currency symbol, and color constants.

database.py — Handles SQLite database initialization and table creation on first run.

db/ — Contains individual database query files for each module (inventory_db.py, sales_db.py, billing_db.py, deliver_db.py, purchases_db.py).

gui/ — Contains individual UI page files (dashboard.py, inventory.py, sales.py, billing.py, deliver.py, reports.py) and reusable widget components.

utils/ — Helper utilities including PDF invoice generator (pdf.py) and Excel export helper (excel.py).

6. Module Description

Module	Description
Dashboard	Displays real-time overview including total stock items, low stock alerts, total revenue, pending deliveries, and recent activity.
Inventory	Allows adding, editing, and deleting raw materials. Tracks stock levels and sends low-stock warnings. Records purchases from suppliers and automatically updates stock quantities.
Sales	Records customer sales with box type, quantity, price per unit, and auto-calculated total. Supports search and delete functionality.
Billing	Generates professional tax invoices with auto-incremented invoice numbers (INV-0001 format). Invoices can be downloaded as PDF files.
Ready to Deliver	Tracks delivery status (Ready, Dispatched, Delivered) for each customer order. Status can be updated directly from the list.
Reports	Generates daily and monthly reports combining stock, sales, and delivery data. Reports can be exported to Excel (.xlsx) format.

7. Key Features

- Real desktop window — no browser required.
- Dark and Light mode toggle from sidebar.
- Search and filter records on every page.
- Delete any record with confirmation dialog.
- PDF invoice generation with professional layout.
- Excel report export for daily and monthly data.

- Auto-incremented invoice numbers (INV-0001, INV-0002...).
- Low stock alerts with visual warning badges.
- Live total calculation in sales and billing forms.
- Delivery status update directly from the list.
- Responsive layout adapting to any screen size (85% of screen).
- Standalone .exe packaging — no Python installation needed.
- Data stored permanently in local SQLite database.

8. Database Design

The application uses SQLite3 as the local database. The following tables are created automatically on first launch:

inventory — id, item_name, quantity, unit, low_stock_threshold

purchases — id, item_id (FK), quantity, cost, supplier_name, date

sales — id, customer_name, box_type, quantity, price_per_unit, total_amount, date

billing — id, invoice_number, customer_name, quantity, price, total_amount, date

ready_to_deliver — id, customer_name, status, date

9. Conclusion

The Factory Manager application successfully achieves its goal of providing a simple, efficient, and reliable desktop solution for small manufacturing businesses. The project demonstrates the practical use of Python for building real-world business applications.

The application covers all key aspects of factory management including inventory tracking, sales recording, invoice generation, delivery monitoring, and report generation. By packaging it as a standalone executable, it can be deployed on any Windows computer without requiring any technical setup.

Future enhancements could include multi-user network support, barcode scanning integration, and cloud database synchronization for businesses with multiple workstations.

10. Future Scope

- Multi-user network support with shared database over LAN.
- Cloud database integration for remote access.
- Barcode/QR code scanning for inventory items.
- Customer management module with contact history.
- Automated WhatsApp/email invoice delivery.
- Android/iOS mobile companion app.
- Graphical analytics dashboard with charts.